CMSC 420: Preliminaries

Justin Wyss-Gallifent

February 16, 2026

1 Problems **2**

1 Problems

1. Calculate and simplify.

Sum	**Result**
(a) $\sum_{i=5}^{105} 7$	707
(b) $\sum_{i=5}^{105} i$	5555
(c) $\sum_{i=n}^{2n} i$ with $n \geq 0$	$\frac{3}{2}n^2 + \frac{3}{2}n$
(d) $\sum_{i=1}^{117} i^2$	540735
(e) $\sum_{i=0}^{7} \left[1 + 2^{3i}\right]$	2396753

2. Suppose that we have a 0-indexed list `A` of length `n`. We have the following pseudocode:

```
function shift(A,n,i):
    for j = i to n-2 inclusive:
        A[j] = A[j+1]
    end for
end function
```

Suppose the code `A[j] = A[j+1]` takes j seconds to execute. If an index $0 \leq i \leq n-1$ is chosen uniformly at random what is the expected time it will take for the code to run?

Note: There are several $\sum$ expressions involved. You should show what these evaluate to but after that you can use software tools to simplify the resulting expressions.

Solution:
For an element at index i we have to shift all elements at indices $i+1 < j \leq n-1$ which may be no elements at all if $i = n-1$. For each j it takes j seconds for a total time of:

$$\sum_{j=i}^{n-2} j = \frac{(n-2)(n-1)}{2} - \frac{(i-1)(i)}{2}$$

Note that if $i = n-1$ then this is 0, which makes sense.
Since there is a $\frac{1}{n}$ probability of choosing each element the expected deletion time is:

$$\begin{aligned}
\sum_{i=0}^{n-1} \frac{1}{n}\left(\frac{(n-2)(n-1)}{2} - \frac{(i-1)(i)}{2}\right) &= \frac{1}{2n}\sum_{i=0}^{n-1}(n-2)(n-1) - \frac{1}{2n}\sum_{i=0}^{n-1}(i-1)(i) \\
&= \frac{1}{2}(n-2)(n-1) - \frac{1}{2n}\sum_{i=0}^{n-1} i^2 + \frac{1}{2n}\sum_{i=0}^{n-1} i \\
&= \frac{1}{2}(n-2)(n-1) - \frac{1}{2n}\left(\frac{(n-1)(n)(2(n-1)+1)}{6}\right) \\
&+ \frac{1}{2n}\left(\frac{(n-1)(n)}{2}\right) \\
&= \frac{1}{3}(n^2 - 3n + 2)
\end{aligned}$$

3. Suppose A is a 0-indexed $n \times n$ array. We choose a coordinate (i, j) with all coordinates equally likely. We delete all elements in row i and all elements in column j. When we do this, all elements in the lower left (if any) shift up (one shift per element), all elements in the upper right (if any) shift left (one shift per element), and all elements in the lower right (if any) shift up and left (two shifts per element). If each shift takes one second how long do we expect the deletion to take? Simplify your final (polynomial) expression.

Solution:
If we delete the element at coordinates (i, j) then:

- There are $(n-i-1)(j)$ elements in the lower left.
- There are $(i)(n-j-1)$ elements in the upper right.
- There are $(n-i-1)(n-j-1)$ elements in the lower right.

Consequently the expected value is:

$$
\begin{aligned}
E &= \frac{1}{n^2} \sum_{i=0}^{n-1} \left[\sum_{j=0}^{n-1} (n-i-1)(j) + (i)(n-j-1) + 2(n-i-1)(n-j-1) \right] \\
&= \frac{1}{n^2} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \left[i(1-n) + j(1-n) + 2n^2 - 4n + 2 \right] \\
&= \frac{1}{n^2} \left[(1-n) \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (i) + (1-n) \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (j) + 2n^4 - 4n^3 + 2n^2 \right] \\
&= \frac{1}{n^2} \left[(1-n)n \frac{(n-1)(n)}{2} + (1-n)n \frac{(n-1)(n)}{2} + 2n^4 - 4n^3 + 2n^2 \right] \\
&= \frac{1}{n^2} \left[(1-n)n(n-1)(n) + 2n^4 - 4n^3 + 2n^2 \right] \\
&= (1-n)(n-1) + 2n^2 - 4n + 2 \\
&= -n^2 + 2n - 1 + 2n^2 - 4n + 2 \\
&= n^2 - 2n + 1
\end{aligned}
$$

4. Solve the following recurrence relation by digging down:

$$T(n) = 2T(n-1) + n \quad \text{with } T(0) = 3$$

Solution:
We have:

$$\begin{aligned}
T(n) &= 2T(n-1) + n \\
&= 2(2T(n-2) + n - 1) + n \\
&= 2^2T(n-2) + 2n + n - 2(1) \\
&= 2^2(2T(n-3) + n - 2) + 2n + n - 2(1) \\
&= 2^3T(n-3) + 4n + 2n + n - 2(1) - 2^2(2) \\
&= 2^3(2T(n-4) + n - 3) + 4n + 2n + n - 2(1) - 2^2(2) \\
&= 2^4T(n-4) + 8n + 4n + 2n + n - 2(1) - 2^2(2) - 2^3(3) \\
&= ... \\
&= 2^kT(n-k) + \sum_{i=0}^{k-1} 2^i n - \sum_{i=1}^{k-1} 2^i i
\end{aligned}$$

This ends when $k = n$ in which case we get:

$$\begin{aligned}
T(n) &= 2^nT(0) + n\sum_{i=0}^{n-1} 2^i - \sum_{i=1}^{n-1} 2^i i \\
&= 2^n(3) + n(2^n - 1) - [(n-2)2^n + 2] \\
&= 2^n(3) + (n)2^n - n - (n)2^n + 2^n(2) - 2 \\
&= 2^n(5) - n - 2
\end{aligned}$$

Note: One of those sums might not be familiar. It's based on:

$$\sum_{i=1}^{n} i2^i = (n-1)2^{n+1} + 2$$

5. Simplify the following sum:

$$1 + \sum_{i=0}^{h-1} 2 \lceil m/2 \rceil^i (\lceil m/2 \rceil - 1)$$

Solution:
We have:

$$\begin{aligned}
1 + \sum_{i=0}^{h-1} 2 \lceil m/2 \rceil^i (\lceil m/2 \rceil - 1) &= 1 + 2(\lceil m/2 \rceil - 1) \sum_{i=0}^{h-1} \lceil m/2 \rceil^i \\
&= 1 + 2(\lceil m/2 \rceil - 1) \left(\frac{\lceil m/2 \rceil^h - 1}{\lceil m/2 \rceil - 1} \right) \\
&= 1 + 2(\lceil m/2 \rceil^h - 1) \\
&= 2 \lceil m/2 \rceil^h - 1
\end{aligned}$$

6. It is the case that $\mathcal{O}(\sqrt{n})$ is slower than $\mathcal{O}(\lg n)$. Formally this means that we have $\lg n = \mathcal{O}(\sqrt{n})$ but $\sqrt{n} \neq \mathcal{O}(\lg n)$.

   (a) State the limit theorem (from CMSC351) which applies to $\mathcal{O}$.

   Solution:
   The limit theorem states that:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} \neq \infty \quad \Longrightarrow \quad f(n) = \mathcal{O}(g(n))$$

   (b) Prove that $\lg n = \mathcal{O}(\sqrt{n})$ using the $\mathcal{O}$ limit theorem.

   Solution:
   We have:

$$\lim_{n\to\infty} \frac{\lg n}{\sqrt{n}} = \lim_{n\to\infty} \frac{1/(n \ln 2)}{1/(2\sqrt{n})} = \lim_{n\to\infty} \frac{2\sqrt{n}}{n \ln 2} = \lim_{n\to\infty} \frac{2}{\ln 2\sqrt{n}} = 0 \neq \infty$$

   The result follows directly from the theorem.

(c) Prove that $\sqrt{n} \neq \mathcal{O}(\lg n)$ using the definition of $\mathcal{O}$.

Note: This is a short but fairly challenging mathematical argument.

Solution 1:

By way of contradiction assume $\sqrt{n} = \mathcal{O}(\lg n)$. This means:

$$\exists C > 0, n_0 \text{ s.t. } \forall n \left[n \geq n_0 \implies \sqrt{n} \leq C \lg n \right]$$

We will find an $n_1 \geq n_0$ such that $\sqrt{n_1} > C \lg(n_1)$, yielding a contradiction.

(i) First, observe that if we choose k so that $2^k > C$ and $2^k > 4k$ then we have:

$$2^{2k} > C(4k)$$
$$\sqrt{2^{4k}} > C \lg(2^{4k})$$

Thus for $n = 2^{4k}$ we have $\sqrt{n} > C \lg n$.

(ii) Second, observe that $\frac{d}{dn}\sqrt{n} = \frac{1}{2\sqrt{n}}$ and $\frac{d}{dn} C \lg n = \frac{C}{n}$ from whence we can conclude that $\frac{1}{2\sqrt{n}} > \frac{C}{n}$ iff $n > 4C^2$.

Consequently for $n > 4C^2$ if $\sqrt{n}$ ever goes above $C \lg n$ then it will stay above since it has a larger slope.

Combining this fact with (i) we can conclude that if $n > \max\{4C^2, 2^{4k}\}$ we can conclude $\sqrt{n} > C \lg n$.

So now consider $n_1 > \max\{4C^2, 2^{4k}, n_0\}$. By (ii) we know that $\sqrt{n}_1 > C \lg n_1$ but also $n_1 \geq n_0$ and so n_1 contradicts the definition.

Solution 2:

By way of contradiction assume $\sqrt{n} = \mathcal{O}(\lg n)$. This means:

$$\exists C > 0, n_0 \text{ s.t. } \forall n \left[n \geq n_0 \implies \sqrt{n} \leq C \lg n \right]$$

Since this is true for all $n \geq n_0$ let's consider $n > 1$ so $\lg n > 0$ and we have:

$$\frac{\sqrt{n}}{\lg n} \leq C$$

Since the inequality holds for all $n > \max(n_0, 1)$ it holds for the limit, too:

$$\lim_{n \to \infty} \frac{\sqrt{n}}{\lg n} \leq C$$

However observe that, rewriting and using L'hôpital's Rule:

$$\lim_{n \to \infty} \frac{n^{1/2}}{\frac{\ln n}{\ln 2}} = \lim_{n \to \infty} \frac{\frac{1}{2} n^{-1/2}}{\frac{1}{n \ln 2}} = \lim_{n \to \infty} \frac{n \ln 2}{2\sqrt{n}} = \lim_{n \to \infty} \frac{\sqrt{n} \ln 2}{2} = \infty$$

This is impossible since C is fixed.

7. Suppose h is a nonnegative integer and we are given that:

$$n \leq \sum_{i=0}^{h} 3^i$$

Find an integer lower bound for h which does not include a sum.

Solution:
We have:

$$n \leq \sum_{i=0}^{h} 3^i = \frac{3^{h+1} - 1}{3 - 1} = \frac{1}{2}\left(3^{h+1} - 1\right)$$

Thus:

$$\begin{aligned}
\frac{1}{2}\left(3^{h+1} - 1\right) &\geq n \\
3^{h+1} - 1 &\geq 2n \\
3^{h+1} &\geq 2n + 1 \\
h + 1 &\geq \log_3(2n + 1) \\
h &\geq \log_3(2n + 1) - 1
\end{aligned}$$

And since h is an integer we have:

$$h \geq \lceil \log_3(2n + 1) - 1 \rceil$$

8. When we talk about *the Ackermann function* in the context that we do, we generally mean the function defined first using:

$$A(0,n) = n+1$$
$$A(m+1,0) = A(m,1)$$
$$A(m+1,n+1) = A(m,A(m+1,n))$$

Then via:

$$Ack(n) = A(n,n)$$

Calculate $Ack(n)$ for $n = 0,1,2$. Show intermediate steps, don't just give the answer.

Solution:
We have:

$$Ack(0) = A(0,0) = 1$$
$$Ack(1) = A(1,1) = A(0,A(1,0)) = A(0,A(0,1)) = A(0,2) = 3$$
$$Ack(2) = ...lol... = 7$$

9. Suppose algorithm A has time complexity $T_A(n) = \Theta(n)$ and algorithm B has time complexity $T_B(n) = \Theta(n^2)$.

   (a) Which algorithm is generally accepted as "faster" and why?

   Solution:
   Algorithm A because the running time is quicker.

   (b) Give an example of two specific running times $T_A(n)$ and $T_B(n)$ such that algorithm B is faster for $0 \leq n < 10000$ and algorithm A is faster for $n > 10000$ and show why this is the case. Along with your example explain how you came up with it, so don't simply give the example. Both should be valid for all $n \geq 0$.

   Solution:
   We choose $T_A(n)$ so it's linear with $T_A(0) = 1$ and we choose $T_B(n)$ so it's quadratic with $T_B(0) = 0$ and we ensure they meet at $n = 10000$. To do this we can set $T_A(n) = n + 1$ and $T_B(n) = bn^2$ with an appropriate b:

$$\begin{aligned} T_A(10000) &= T_B(10000) \\ 10001 &= b(10000^2) \\ b &= \frac{10001}{100000000} \end{aligned}$$

   Thus we have:

$$\begin{aligned} T_A(n) &= n + 1 \\ T_B(n) &= \frac{10001}{100000000} n^2 \end{aligned}$$

10. We wish to store a collection of n key-value pairs where the keys are integers and the values are strings. We put the key-value pairs in a 0-indexed list of objects where each object represents a pair and such that the keys are in increasing order. For example the following key-value pairs:

$$[3, apple],\ [10, cat],\ [5, math],\ [-2, hello]$$

Would be stored in a list A with:

$$A[0] = [-2, hello],\ A[1] = [3, apple],\ A[2] = [5, math],\ A[3] = [10, cat].$$

(a) To search for a particular key we will use a binary search. What is the best and worst case $\mathcal{O}$ time complexity of this? Explain.

Solution:
Best-case is $\mathcal{O}(1)$ when the key is found immediately and the worst-case is $\mathcal{O}(\lg n)$ when the algorithm must iterate the maximum number of times.

(b) To insert a key-value pair we use a binary search to find the next smallest key (do you know how to do this?) and shift to make space accordingly. What is the best and worst case $\mathcal{O}$ time complexity of this? Explain.

Solution:
In the best-case the next smallest key is the last key in which case it takes $\mathcal{O}(\lg n)$ to find that next smallest key and $\mathcal{O}(1)$ to shift to make space for a total of $\mathcal{O}(\lg n)$.
In the worst-case the next smallest key is the middle key in which case it takes $\mathcal{O}(1)$ to find that next smallest key and $\mathcal{O}(n/2) = \mathcal{O}(n)$ to shift to make space for a total of $\mathcal{O}(n)$.

(c) To delete a key-value pair we use a binary search to find the key and then splice to remove the key-value pair accordingly. What is the best and worst case $\mathcal{O}$ time complexity of this? Explain.

Solution:
In the best-case the key is the last key in which case it takes $\mathcal{O}(\lg n)$ to find that key and $\mathcal{O}(1)$ to splice for a total of $\mathcal{O}(\lg n)$.
In the worst-case the key is the middle key in which case it takes $\mathcal{O}(1)$ to find that key and $\mathcal{O}(n/2) = \mathcal{O}(n)$ to splice for a total of $\mathcal{O}(n)$.

11. One of the most frustrating things when it comes to proofs related to algorithms is the fact that floors and ceilings show up and get in the way. Prove that for all $x \geq 2$ we have:

$$\left\lfloor \log_{3/2}(x+1) \right\rfloor - \left\lfloor \log_{3/2} x \right\rfloor \leq 1$$

Note 1: This is not particularly difficult if you're comfortable with some basic calculus so if you find your solution getting a little out of hand you're probably doing more than you need to!

Solution:
Define:

$$f(x) = \log_{3/2}(x+1) - \log_{3/2} x$$

We proceed as follows:

(a) Observe that:

$$f(2) = \log_{3/2} 3 - \log_{3/2} 2 = \log_{3/2}\left(\frac{3}{2}\right) = 1$$

(b) Observe that:

$$f'(x) = \frac{1}{\ln(3/2)}\left(\frac{1}{x+1}\right) - \frac{1}{\ln(3/2)}\left(\frac{1}{x}\right)$$

Since we have $1/x > 1/(x+1)$ we know that $f'(x) < 0$ and so $f(x)$ is decreasing.

(c) From (a) and (b) we know that $f(x) \leq 1$ for all $x \geq 2$ and so:

$$\begin{aligned}
\log_{3/2}(x+1) - \log_{3/2} x &\leq 1 \\
\log_{3/2}(x+1) &\leq 1 + \log_{3/2} x \\
\left\lfloor \log_{3/2}(x+1) \right\rfloor &\leq \left\lfloor 1 + \log_{3/2} x \right\rfloor \\
\left\lfloor \log_{3/2}(x+1) \right\rfloor &\leq \left\lfloor \log_{3/2} x \right\rfloor + 1 \\
\left\lfloor \log_{3/2}(x+1) \right\rfloor - \left\lfloor \log_{3/2} x \right\rfloor &\leq 1
\end{aligned}$$

12. If algorithm A has time complexity $\mathcal{O}(n \lg n)$ and algorithm B has time complexity $\mathcal{O}(n^2)$ does this mean we should always use algorithm A? Explain in no more than one paragraph!

    Solution:
    Time complexity is predicated upon large values of n so all we can say is that for sufficiently large values of n we know algorithm A is faster. However we don't know exactly how bit "sufficiently large" is and so it's certainly possible that for the values of n we are dealing with that algorithm B is faster.

13. Suppose A is a 0-indexed list of integers which has the property that if A has length j then inserting an integer at index $0 \leq k \leq j$ (where $k = j$ is an append) takes time $j - k + 1$ seconds.

(a) Starting with a list of length 0, how many seconds will it take to append n integers?

Solution:
The time required will be $1 + ... + 1 = n$.

(b) Starting with a list of length n, if an integer is inserted into a random index $0 \leq k \leq n$ (with each index equally likely) what is the expected amount of time required? Simplify your answer.

Solution:
The expected time required will be:

$$\begin{aligned}
\sum_{k=0}^{n} \frac{1}{n+1}(n-k+1) &= \frac{1}{n+1}\left[\sum_{k=0}^{n} n - \sum_{k=0}^{n} k + \sum_{k=0}^{n} 1\right] \\
&= \frac{1}{n+1}\left[n(n+1) - \frac{(n)(n+1)}{2} + (n+1)\right] \\
&= n - \frac{1}{2}n + 1 \\
&= \frac{1}{2}n + 1
\end{aligned}$$

14. For each of the following recurrence relations with base cases given, if we dig down k steps calculate which k (as an expression involving n) would yield the base case. The assumption on n is given which guarantees the base case is always reached.

Relation	Assumption on n	k expression
$T(n) = 2T(n/3) + n$ with $T(1) = 3$	$n \in \{1, 3, 9, 27, ...\}$	$\log_3 n$
$T(n) = 3T(n-3)$ with $T(0) = 5$	$n \in \{0, 3, 6, 9, 12, ...\}$	$n/3$
$T(n) = T(n-1) + n^2 + n$ with $T(2) = 7$	$n \in \{2, 3, 4, ...\}$	$n - 2$
$T(n) = 2T(n-2)$ with $T(1) = 10$	$n \in \{1, 3, 5, 7, 9, ...\}$	$(n-1)/2$
$T(n) = aT(n/b) + cn$ with $T(1) = 2$	$n \in \{1, b, b^2, b^3, ...\}$	$\log_b n$
$T(n) = 4T(n/2 - 1)$ with $T(0) = 17$	$n \in \{2, 6, 14, 30, ...\}$	$\lg(n+2) - 1$

15. Calculate each of the following permutations or combinations and simplify.

Problem	Answer
$P(10, 3)$	720
$C(10, 3)$	120
$P(7, 7)$	5040
$C(7, 7)$	1

16. For each of the following, some fact about a recurrence relation is given. From that fact what $\mathcal{O}$, Ω, or Θ fact can you conclude about $T(n)$? Draw the strongest conclusion possible.

Fact	Conclusion about $T(n)$
(a) $T(n) = n \lfloor \lg n + 10 \rfloor$	$\Theta(n \lg n)$
(b) $T(n) = 4T(n/4) + n$	$\Theta(n \lg n)$
(c) $T(n) \geq n^2 + 2n + 1$	$\Omega(n^2)$
(d) $T(n) \leq \frac{1}{n}$	$\mathcal{O}(1)$
(e) $T(n) \leq n\lvert \sin(n) \rvert$	$\mathcal{O}(n)$

17. Calculate each of the following and simplify:

Calculation	Result
(a) $\sum_{i=1}^{10} i$	55
(b) $P(8,2)$	56
(c) $C(8,2)$	28

18. Suppose that we have a 0-indexed list `A` of length `n`. We have the following pseudocode:

```
function zero(A,n,i):
    for j = 0 to i inclusive:
        A[j] = 0
    end for
end function
```

Suppose the code `A[j] = 0` takes j seconds to execute. If an index $0 \leq i \leq n-1$ is chosen uniformly at random what is the expected time it will take for the code to run? You can (should) leave any sums un-evaluated in your answer.

Time =	$\sum_{i=0}^{n-1} \left(\frac{1}{n} \sum_{j=0}^{i} j \right)$

19. For each of the following, some fact about a function $T(n)$ is given. From that fact what $\mathcal{O}$, Ω, or Θ fact can you conclude about $T(n)$? Draw the strongest conclusion possible.

Answers should have the form $\mathcal{O}(g(n))$ or $\Theta(g(n))$ or $\Omega(g(n))$ for some nice function $g(n)$.

Fact	Conclusion about $T(n)$
(a) $T(n) \leq 2n \lg n + n + 1$	$\mathcal{O}(n \lg n)$
(b) $T(n) = 2T(n/2) + n + 1$	$\Theta(n \lg n)$
(c) $T(n) \geq 0.01n + \sqrt{n}$	$\Omega(n)$
(d) $T(n) = 3T(n/27) + \sqrt{n}$	$\Theta\left(\sqrt{n}\right)$
(e) $T(n) = 3n + \lfloor n/10 \rfloor \left(\lceil n/10 \rceil + 1\right)$	$\Theta\left(n^2\right)$

20. Suppose n is a positive integer. Find an expression in terms of n for the...

Criteria	Result
...largest $k \in \mathcal{Z}$ Such That $2^k \leq n$	$\lfloor \lg n \rfloor$
...smallest $k \in \mathcal{Z}$ Such That $2^{k+1} \geq n$	$\lceil \lg(n) - 1 \rceil$
...largest $k \in \mathcal{Z}$ Such That $\log_2 k \leq n$	$\lfloor 2^n \rfloor$

21. Suppose you roll a 6-sided die until a 6 appears and then you count the number of rolls which occurred before this 6 appeared.

(a) What is the probability that there were no rolls before the first 6?

Solution:
The probability is $1/6$.

(b) What is the probability that there was exactly one roll before the first 6?

Solution:
The probability is $(5/6)(1/6) = 5/36$.

(c) What is the probability that there were either two or fewer rolls before the first 6?

Solution:
The probability is:

$$(1/6) + (5/6)(1/6) + (5/6)^2(1/6) = 91/216$$

(d) What is the probability that there were three or more rolls before the first 6?

Solution:
The probability is $1 - 91/216 = 125/216$.
Note: This could also be done by explicitly counting the associated probabilities but it is a bit more work and requires remembering the sum of an infinite series:

$$\begin{aligned}
(5/6)^3(1/6) + (5/6)^4(1/6) + ... &= \sum_{i=3}^{\infty}(5/6)^i(1/6) \\
&= (5/6)^3(1/6)\sum_{i=0}^{\infty}(5/6)^i \\
&= (5/6)^3(1/6)\left(\frac{1}{1-5/6}\right) \\
&= (5/6)^3(1/6)(6) \\
&= 125/216
\end{aligned}$$

22. Solve the following recurrence relation by digging down:

$$T(n) = T(n/2) + 1 \quad \text{with } T(1) = 3$$

Solution:
We have:

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &= T(n/4) + 2 \\ &= ... \\ &= T(n/2^k) + k \end{aligned}$$

We reach the base case when $n/2^k = 1$ so $k = \lg n$ and so we have:

$$T(n) = T(1) + \lg n = 3 + \lg n$$

23. Consider the following recurrence relation:

$$T(n) \leq T(n-2) + 1 \quad \text{with } T(0) \leq 10 \text{ and } T(1) \leq 20$$

(a) Dig down to draw a conclusion about $T(n)$ for the case where n is even.

Solution:
The fact that n is even guarantees we reach the base case $T(0)$.
We have:

$$\begin{aligned} T(n) &\leq T(n-2) + 1 \\ &\leq T(n-4) + 2 \\ &\leq ... \\ &\leq T(0) + \frac{n}{2} \\ &\leq 10 + \frac{n}{2} \end{aligned}$$

(b) Dig down to draw a conclusion about $T(n)$ for the case where n is odd.

Solution:
The fact that n is odd guarantees we reach the base case $T(1)$ but we have to be a bit careful. It might help to find the general form:
We have:

$$\begin{aligned} T(n) &\leq T(n-2) + 1 \\ &\leq T(n-4) + 2 \\ &\leq ... \\ &\leq T(n-2k) + k \end{aligned}$$

Since n is odd we reach the base case $T(1)$ when $n - 2k = 1$, so $k = \frac{n-1}{2}$, yielding:

$$T(n) \leq T(1) + \frac{n-1}{2} \leq 20 + \frac{n-1}{2}$$

(c) What $\mathcal{O}$ fact can we conclude about $T(n)$?

Solution:
We can conclude that $T(n) = \mathcal{O}(n)$.

24. Consider the following recurrence relation:

$$T(n) \leq 2T(n-3) + 1 \quad \text{with } T(0) = 1 \text{ and } T(1) = 2 \text{ and } T(2) = 3$$

Dig down to draw a conclusion about $T(n)$ for the case where n is a multiple of 3.

Solution:
The fact that n is a multiple of 3 guarantees we reach the base case $T(0)$. We have:

$$\begin{aligned}
T(n) &\leq 2T(n-3) + 1 \\
&\leq 2(2T(n-6) + 1) + 1 = 2^2T(n-6) + 2 + 1 \\
&\leq 2^2(2T(n-9) + 1) + 2 + 1 = 2^3T(n-9) + 4 + 2 + 1 \\
&\leq ... \\
&\leq 2^kT(n-3k) + \sum_{i=0}^{k-1} 2^i
\end{aligned}$$

We reach the base case when $k = n/3$ and so we have:

$$T(n) \leq 2^{n/3}T(0) + \sum_{i=0}^{n/3-1} 2^i = 2^{n/3}(1) + 2^{n/3} - 1 = 2 \cdot 2^{n/3} - 1$$

Note that we could clean this up a bit and see what $\mathcal{O}$ would be:

$$T(n) \leq 2 \cdot (2^{1/3})^n - 1 = 2(\sqrt[3]{2})^n - 1$$

Thus:

$$T(n) = \mathcal{O}\left((\sqrt[3]{2})^n\right)$$

25. Suppose n is an unknown positive integer and you wish to add all powers of 2 which are less than or equal to n

(a) Write down a closed form for this sum.

Solution:
We wish to calculate:

$$1 + 2 + 4 + ... + 2^k$$

where 2^k is the largest power of 2 less than or equal to n, meaning $2^k \leq n$, or $k \leq \lg n$. The largest of these will be $k = \lfloor \lg n \rfloor$ so our sum is:

$$\sum_{i=0}^{\lfloor \lg n \rfloor} 2^i = 2^{\lfloor \lg n \rfloor + 1} - 1$$

(b) What nice $\mathcal{O}$ fact can you conclude about this?

Solution:
In general $\lfloor x \rfloor \leq x$ and so here:

$$2^{\lfloor \lg n \rfloor} - 1 \leq 2^{\lg n} - 1 = n - 1$$

Thus our expression is $\mathcal{O}(n)$.

26. Calculate the following sum, showing steps:

$$\sum_{i=1}^{10} 5^{6-i}$$

Solution:
We have:

$$\begin{aligned}
\sum_{i=1}^{10} 5^{6-i} &= \left(\sum_{i=0}^{10} 5^{6-i}\right) - 5^{6-0} \\
&= \left(5^6 \sum_{i=0}^{10} 5^{-i}\right) - 5^6 \\
&= \left(5^6 \sum_{i=0}^{10} (1/5)^i\right) - 5^6 \\
&= 5^6 \left(\frac{1-(1/5)^{11}}{1-(1/5)}\right) - 5^6
\end{aligned}$$

27. Calculate the following sum, showing steps:

$$\sum_{i=3}^{n} 7^{2i+3}$$

Solution:
We have:

$$\begin{aligned}
\sum_{i=3}^{n} 7^{2i+3} &= \left(\sum_{i=0}^{n} 7^{2i+3}\right) - 7^{2(0)+3} - 7^{2(1)+3} - 7^{2(2)+3} \\
&= 7^3 \left(\sum_{i=0}^{n} 7^{2i}\right) - 7^3 - 7^5 - 7^7 \\
&= 7^3 \left(\sum_{i=0}^{n} 49^{i}\right) - 7^3 - 7^5 - 7^7 \\
&= 7^3 \left(\frac{49^{n+1} - 1}{49 - 1}\right) - 7^3 - 7^5 - 7^7
\end{aligned}$$